A Comparative Evaluation of Per-Token Log-Likelihood and Curvature-Based Zero-Shot Detection of LLM-Generated Text

Deogracias Mufula
StudentNumber:MFLDEO001
Department of Computer Science, University of Cape Town

Abstract

Zero-shot statistical detectors offer a practical way of identifying LLM-generated text without requiring labelled training data, but the relative effectiveness of two established approaches, namely per-token log-likelihood thresholding and curvature-based (DetectGPT-style) scoring, has not been systematically compared under a single controlled framework. We evaluate both detectors on a paired dataset of 500 XSum-derived news articles and Gemma 3 4B-generated versions of the same articles, using Llama 3.2 3B as a common scoring backbone with thresholds calibrated on a validation set. Log-likelihood substantially outperformed curvature (AUROC 0.9524 vs. 0.6668, $p < 10^{-5}$ by both McNemar's and DeLong's tests), a result that held even when we re-ran curvature with the originally proposed T5-large perturbation model instead of the T5-base substitution used in our primary experiments (AUROC 0.6937), indicating the gap is not primarily attributable to perturbation model size. A secondary logistic-regression ensemble combining both signals achieved a high absolute AUROC (0.9380) but did not exceed log-likelihood alone. We additionally report two TF-IDF logistic regression baselines for reference: one trained directly on our dataset (AUROC 0.9962) and one applied under domain transfer from a separately-trained shared model (AUROC 0.9719), both exceeding the zero-shot detectors. Our results suggest that, in this controlled setting, the additional computational expense of perturbation-based curvature scoring is not justified by its detection performance relative to log-likelihood.

Keywords

LLM-generated text detection, zero-shot detection, log-likelihood, curvature, DetectGPT, natural language processing

1 Introduction

Large language models can now produce fluent, coherent text that is often indistinguishable from human writing [15]. Human readers are themselves unreliable judges of this distinction: Ippolito et al. [4] show that untrained human raters perform only marginally better than chance at identifying machine-generated text, and that this ability degrades further as generation quality improves, underscoring the need for automated methods that do not rely on human judgement. This capability raises serious concerns for academic integrity, journalism, and the broader trustworthiness of online content [10, 14]. As generative models continue to improve, the need for reliable methods to distinguish human-written from machine-generated text has become increasingly urgent.

Zero-shot statistical detectors offer a practical response to this need, as they require no labelled training data and can be applied directly to the output of any language model whose probabilities are accessible [2, 11]. Two of the most established approaches in this space are per-token log-likelihood thresholding [11] and curvature-based scoring, popularised by DetectGPT [7]. While both methods have been evaluated independently in prior work, their relative effectiveness has not been systematically compared under a single, controlled experimental framework using the same dataset, generator, and scoring backbone. Both families of detector, moreover, remain vulnerable to adversarial paraphrasing, which has been shown to substantially degrade zero-shot detector performance [5]; we return to this vulnerability directly in our own robustness evaluation (Section 4.5).

This paper addresses that gap directly. We implement and evaluate both detectors under identical conditions: the same paired human/machine dataset, the same fixed scoring backbone, and the same evaluation protocol. Our research question is: how well do per token log-likelihood and curvature-based scoring compare when detecting machine-generated text, generated from human-written news articles, under the same experimental conditions?

Our contributions are threefold. First, we provide a controlled, paired comparison of log-likelihood and curvature-based detection on XSum-derived human text against Gemma 3 4B-generated counterparts [3], using Llama 3.2 3B as a single fixed scoring backbone. Second, we analyse the computational cost of curvature-based scoring against its accuracy, directly addressing whether the added cost of perturbation-based scoring is justified. Third, as a secondary contribution, we evaluate whether combining both signals through a lightweight logistic-regression ensemble improves on either detector individually.

The remainder of this paper is organised as follows: Section 2 reviews the relevant literature on zero-shot statistical detection; Section 3 describes our dataset, detectors, and evaluation methodology; Section 4 presents our results; Section 5 discusses their implications; Section 6 outlines the limitations of this work; and Section 7 concludes.

2 Related Work

2.1 Per-Token Log-Likelihood

Zero-shot statistical detection methods exploit measurable properties of how language models generate text, requiring no labelled training data. One of the simplest such properties is log-likelihood: the probability a language model assigns to each token in a sequence, conditioned on the tokens preceding it. Solaiman et al. [11] established log-likelihood *thresholding* as one of the earliest such baselines, that is, computing this average probability for a candidate text and classifying it as machine-generated if the value exceeds a fixed cutoff. This works because machine-generated text tends toward higher average per-token log-probability, since language models produce tokens that are statistically likely given their preceding context. This simple signal remains a competitive baseline

against which more sophisticated methods continue to be benchmarked. Gehrmann et al. [2] extended this line of work with GLTR, a visual forensic tool that highlights tokens by their likelihood under a language model, making the log-likelihood signal interpretable to human reviewers rather than reducing it to a single score.

2.2 Curvature-Based Detection (DetectGPT)

Curvature refers to how sharply a text's probability under a language model changes as the text is slightly perturbed. If a text sits at a sharp local peak of the model's probability landscape, small perturbations will cause its probability to drop noticeably; if it sits in a flatter region, perturbations will cause little change. Mitchell et al. [7] introduced DetectGPT, proposing that machine-generated text occupies such sharp local peaks more consistently than human-written text does. DetectGPT uses this idea by comparing the log-probability of a given text with the log-probabilities of several meaning-preserving variations produced by a mask-filling model. If there is a large difference between the original text and these variations, this is taken as an indication that the text was generated by a machine. On controlled benchmarks, DetectGPT achieved strong AUROC scores and performed better than simply using a log-likelihood threshold.

However, later studies have raised concerns about how well these results carry over to real-world settings. Bao et al. [1] proposed Fast-DetectGPT, which removes the need to generate external perturbations and instead uses a more efficient approach based on conditional probability curvature. This change was mainly motivated by the high computational cost of generating and scoring multiple perturbations for each candidate text, which is also a cost considered in this study. Using the DetectRL benchmark, Wu et al. [14] found that both DetectGPT and Fast-DetectGPT perform considerably worse under real-world conditions. In particular, DetectGPT achieved an AUROC of only 22.15% for academic writing and 12.21% for news writing, with the latter being especially relevant to the news-based dataset used in this study.

2.3 Research Gap

Despite this body of work, no study has directly compared per-token log-likelihood and curvature-based scoring under a single controlled framework, same dataset, same generator, and critically, the same fixed scoring backbone. Existing comparisons typically draw on results reported under different experimental conditions, making it difficult to isolate whether performance differences arise from the detection method itself or from confounding factors such as backbone choice or dataset domain. Broader benchmarking efforts such as TuringBench [13] similarly evaluate detection across multiple generators, but do not isolate the effect of detector *type* under a fixed scoring model, the gap this paper targets. This paper addresses that gap directly; furthermore, it directly influenced our change of dataset in order to evaluate the conditions under a shared domain.

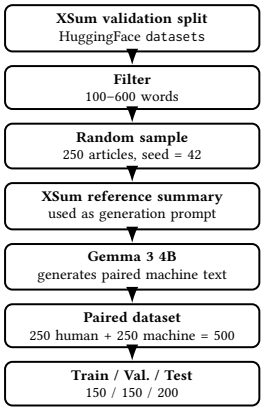


Figure 1: Dataset construction pipeline. Human-written articles are sampled from the XSum validation split and filtered by length; each article's reference summary is used as a generation prompt for Gemma 3 4B to produce a paired machine-generated counterpart, yielding a topic-matched dataset split into training, validation, and test sets.

3 Methodology

3.1 Dataset

We construct our dataset from XSum [8], a corpus of BBC News articles paired with single-sentence reference summaries. We download the XSum validation split via the HuggingFace `datasets` library and filter articles to those between 100 and 600 words, discarding shorter texts that yield noisy log-likelihood estimates and longer texts that make curvature scoring, which requires 30 perturbations per sample, inherently slow. From the filtered set, we randomly sample 250 articles (seed = 42) as our human-written class.

For each selected article, we use its XSum reference summary as a generation prompt for Gemma 3 4B, producing a paired machine generated counterpart. This yields 500 samples in total: 250 human-written and 250 machine-generated, each machine sample paired with exactly one human source article. We split the dataset into training, validation, and test sets of 150, 150, and 200 samples respectively, each maintaining an equal balance of human and machine samples (Table 1).

This differs from our original proposal, which planned to use Common Crawl as the source of human-written text. Instead, we use XSum because it provides a well-documented and reproducible dataset. The main idea is that the human-written texts are limited to BBC News articles in order to maintain a singular domain rather than representing a wider range of web content which would consist of a wider domain.

Although the test split contains 200 samples, each sample is a full-length news article rather than a short snippet. Articles were filtered to between 100 and 600 words (Section 3), with a mean length of 294.24 words with the highest length article having 596 words across the human written articles dataset, and Gemma 3 4B's generated counterparts followed a comparable length distribution (306.59 words on average with the longest LLM generated article having 345 words). The 500-sample dataset therefore corresponds to 150,208 words of scored text in total (73,560 words from the human-written

Table 1: Dataset composition by split.

Split	Human	Machine	Total
Train	75	75	150
Validation	75	75	150
Test	100	100	200
Total	**250**	**250**	**500**

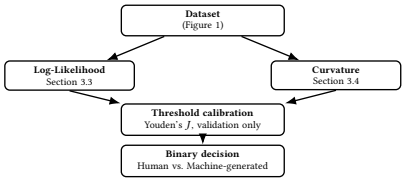


Figure 2: Detection methodology overview. Both detectors score the same dataset using Llama 3.2 3B as a shared backbone; a threshold is selected on the validation split only (Section 3.5) and applied to produce a binary decision.

subset and 76,648 words from the machine-generated subset), and the apparent smallness of the sample count should be read against this article-level, rather than sentence-level, unit of analysis.

3.2 Detection Pipeline Overview

Figure 2 summarises the common evaluation pipeline used by both detectors. The two scoring methods operate on the same dataset and share Llama 3.2 3B as the scoring backbone.

3.3 Per-Token Log-Likelihood

Per-token log-likelihood thresholding scores a candidate text by the average probability a language model assigns to each token, given the tokens preceding it. For a text $x = (t_1, \ldots, t_n)$ and a scoring language model p_θ:

$$\mathcal{L}(x) = \frac{1}{n} \sum_{i=1}^{n} \log p_\theta(t_i \mid t_1, \ldots, t_{i-1}) \tag{1}$$

Machine-generated text tends toward a higher (less negative) value of $\mathcal{L}(x)$, since language models generate text by selecting statistically likely tokens given their context, whereas human writing is not constrained in this way [11]. The first token in a sequence has no preceding context and is excluded from scoring. A candidate text is classified as machine-generated if $\mathcal{L}(x)$ exceeds a threshold τ_{LL}, selected as described in Section 3.5.

3.4 Curvature-Based (DetectGPT-Style) Detection

Curvature-based detection, following Mitchell et al. [7], measures a different property: not how probable a text is in isolation, but how much its probability exceeds that of small, meaning-preserving perturbations of itself. Intuitively, log-likelihood asks *how probable is the observed text*, while curvature asks *how unusually high is the observed text's probability relative to nearby variations of it*. For a text x, k perturbations $\tilde{x}_j$ are generated by masking random spans

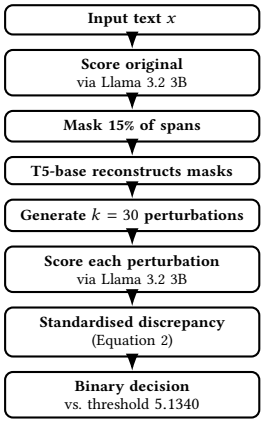


Figure 3: Curvature-based (DetectGPT-style) detection pipeline. The input text is first scored using Llama 3.2 3B; T5-base then masks and reconstructs 15% of the text's spans, repeated to generate 30 perturbations, each rescored by the same backbone. The standardised discrepancy between the original and perturbation scores (Equation 2) is compared against the validation-calibrated threshold to produce the final decision.

and reconstructing them with a mask-filling model, then both x and each $\tilde{x}_j$ are scored under the same scoring model:

$$d(x) = \frac{\log p_\theta(x) - \frac{1}{k}\sum_{j=1}^{k} \log p_\theta(\tilde{x}_j)}{\sigma\left(\{\log p_\theta(\tilde{x}_j)\}_{j=1}^{k}\right)} \tag{2}$$

where the numerator is the gap between the original text's log-probability and the mean log-probability of its perturbations, and the denominator is the standard deviation of the perturbation log-probabilities, matching DetectGPT's original standardised formulation. This corrects an earlier, unnormalised implementation used during development (see Section 6). A large positive $d(x)$ indicates the original text sits at a sharper local peak of the model's probability landscape than its perturbed neighbours, which Mitchell et al. hypothesise is more characteristic of machine-generated text. A candidate is classified as machine-generated if $d(x)$ exceeds a threshold τ_{curv} (Section 3.5). Figure 3 illustrates this process in three stages.

3.5 Experimental Setup

We use Llama 3.2 3B [12] as the scoring backbone p_θ for both detectors. For curvature, we use T5 [9] as the mask-filling perturbation model, with $k = 30$ perturbations per sample generated by masking random spans covering 15% of each text. Our proposal originally specified T5-large as the perturbation model; due to GPU memory and runtime constraints on the compute infrastructure available to us, our primary experiments substitute T5-base. Given additional compute access later in the project, we also repeated curvature scoring using T5-large as a direct test of this substitution's impact; both results are reported in Section 4 and compared in Table 10.
For both detectors, texts shorter than 10 tokens are excluded, as they produce unreliable score estimates. The decision threshold

Table 2: Experimental configuration for both detectors.

Parameter	Log-Likelihood	Curvature
Scoring model	Llama 3.2 3B	Llama 3.2 3B
Perturbation model	–	T5-base / T5-large
Perturbations (k)	–	30
Mask ratio	–	0.15
Min. tokens	10	10
Threshold criterion	Youden's J	Youden's J
Threshold split	Validation	Validation
Bootstrap replicates	1000	1000

is selected on the validation split only, using Youden's J statistic (sensitivity + specificity -1), maximised over all candidate thresholds observed in the validation scores; the test split is never used for threshold selection. This procedure yielded $\tau_{\text{LL}} = -2.0031$ and $\tau_{\text{curv}} = 5.1340$ (T5-base) or 4.4115 (T5-large). Table 2 summarises the full experimental configuration.

3.6 Evaluation Protocol

We report five metrics on the held-out test split. AUROC (area under the ROC curve) measures a detector's ability to rank a randomly chosen machine-generated text above a randomly chosen human-written text, independent of any specific threshold; we use it as our primary metric for comparing detectors, since it is not sensitive to the particular operating point chosen. F1, precision, recall, and accuracy are reported at each detector's calibrated threshold (Section 3.5) to characterise its behaviour as an actual binary classifier: precision reflects how often a machine-generated prediction is correct (its inverse relates directly to false accusations against human authors), recall reflects how much machine-generated text is actually caught, and accuracy summarises overall correctness across both classes. To quantify uncertainty, we compute 95% confidence intervals for AUROC via bootstrap resampling: we resample the test set with replacement 1000 times (seed = 42), recompute AUROC on each resample, and report the 2.5th and 97.5th percentiles of the resulting distribution.

3.7 Ensemble (Secondary Analysis)

As a secondary analysis, we investigate whether the two detectors' signals are complementary by training a logistic regression classifier over standardised log-likelihood and curvature scores. We fit this ensemble on a stratified 50% split of the full 500-sample dataset and evaluate on the held-out remaining 50%, avoiding an optimistic estimate that would result from fitting and evaluating on the same data.

3.8 TF-IDF Logistic Regression Baseline

As a further point of reference, we evaluate a TF-IDF logistic regression baseline, a supervised approach rather than a zero-shot method, in two variants. The first is trained directly on our own training split (150 samples): we fit a TF-IDF vectorizer with unigram and bigram features (10,000 maximum features, English stop-word removal) on the training texts, then train a logistic regression classifier on the resulting representation, evaluating on the same held-out test split used for both zero-shot detectors. The second variant uses a pre-trained TF-IDF logistic regression model supplied by a collaborator on this project, trained on a separate, group-shared dataset generated using ChatGPT, GPT-4, GPT-2, Mistral, and Llama; we apply this model, unmodified, to our own held-out test set as a domain-transfer evaluation. Unlike the zero-shot detectors, both TF-IDF variants require labelled training data, and their inclusion allows us to contextualise the zero-shot detectors' performance against a supervised alternative.

Table 3: Comparative detector performance on the held-out test set. Curvature is reported for both the primary T5-base perturbation model and the T5-large follow-up test (Section 5).

Detector	AUROC	F1	Precision	Recall	Accuracy
Log-Likelihood	0.9524	0.8889	0.8276	0.9600	0.8800
Curvature (T5-base)	0.6668	0.7645	0.6226	0.9900	0.6950
Curvature (T5-large)	0.6937	0.7645	0.6226	0.9900	0.6950
Shared TF-IDF Baseline	0.9719	0.8475	0.9740	0.7500	0.8650
Self-Trained TF-IDF (in-domain)	0.9962	0.9641	0.9895	0.9400	0.9650

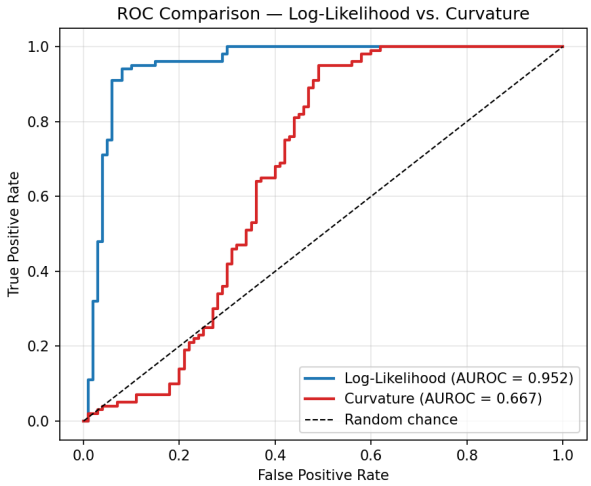


Figure 4: ROC curves for log-likelihood and curvature on the held-out test set, with AUROC values annotated in the legend. The closer a curve sits to the top-left corner, the stronger the detector's ability to distinguish human from machine-generated text; log-likelihood's curve dominates curvature's across nearly the entire range of thresholds.

4 Results

4.1 Comparative Detector Performance

4.2 Confusion Matrix Analysis

On the held-out test set, log-likelihood produced 96 true positives, 80 true negatives, 20 false positives, and 4 false negatives. Curvature, under both T5-base and T5-large, produced 99 true positives, 40 true negatives, 60 false positives, and 1 false negative, an identical confusion matrix despite different underlying scores (see Table 10

Table 4: Bootstrap AUROC confidence intervals and pairwise significance tests.

Detector	95% Bootstrap CI	McNemar p	DeLong p
Log-Likelihood	[0.9135, 0.9831]	5.74×10^{-6}	3.19×10^{-6}
Curvature (T5-base)	[0.5896, 0.7482]		

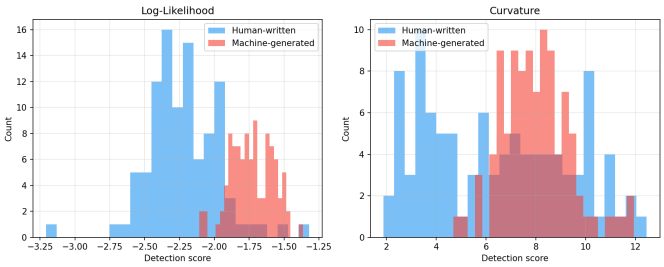


Figure 5: Distribution of detection scores for human-written (blue) and machine-generated (red) test samples, for log-likelihood and curvature respectively. Greater separation between the two distributions indicates stronger discriminative power; the substantial overlap visible for curvature explains its lower precision reported in Table 3.

for the AUROC comparison). The self-trained TF-IDF baseline produced 93 true positives, 99 true negatives, 1 false positive, and 7 false negatives, while the shared, domain-transfer TF-IDF baseline produced 75 true positives, 98 true negatives, 2 false positives, and 25 false negatives. Curvature's low precision is driven by 60 false positives out of 100 human samples, meaning more than half of all human-written text in the test set was misclassified as machine-generated. Log-likelihood's errors are more balanced, while the shared TF-IDF baseline shows the opposite pattern to curvature: very few false positives but a substantially higher false negative count, consistent with its conservative behaviour under domain transfer discussed in Section 5.

4.3 Statistical Significance

Both tests indicate the observed performance difference between the two detectors is statistically significant. McNemar's test on the discordant prediction pairs ($b = 50$, $c = 13$) confirms log-likelihood makes substantially fewer classification errors than curvature on cases where the two detectors disagree ($\chi^2 = 20.57$, $p = 5.74 \times 10^{-6}$). DeLong's test on the underlying AUROC scores corroborates this, confirming the 0.2856 AUROC gap is statistically significant ($z = 4.66$, $p = 3.19 \times 10^{-6}$).

4.4 Qualitative Prediction Disagreements

Of the 63 samples where the two detectors disagreed, the dominant pattern (49 of 63, 78%) is log-likelihood correctly identifying human-written text that curvature misclassifies as machine-generated. Table 5 shows five representative examples of this pattern, each drawn from the XSum test split.

Table 5: Representative XSum test articles where log-likelihood correctly classified human-written text that curvature misclassified as machine-generated.

Article ID	Excerpt	LL	Curv.
`xsum_38902480`	"The Met, the largest force, topped the list, spending £5.2m from 2011-16..."	Human	Machine
`xsum_22699705`	"The building suffers from flooding, has outdated electrical and drainage infrastructure..."	Human	Machine
`xsum_36396341`	"Andrew Nay, 39, crashed his Land Rover into the girls' family car..."	Human	Machine
`xsum_34744458`	"Sylvia Bradshaw, of Fakenham, Norfolk, heard an appeal by Gamble Plant Skip Hire..."	Human	Machine
`xsum_37805755`	"It follows concerns changes proposed by the UK's Food Standards Agency..."	Human	Machine

In article `xsum_38902480`, which reports on UK police forces' spending on informants, curvature flags the text as machine-generated despite its dry, procedural style, quoting attributed sources and official statements, exactly the kind of grounded factual reporting that characterises human BBC journalism. Article `xsum_22699705`, describing structural defects in the Palace of Westminster, shows a similar pattern: specific figures (£1.5bn), named parties (the Commons, the Lords), and a concrete timeline are all present, yet curvature still misclassifies it. Article `xsum_36396341`, reporting a car crash and its victims by name, and article `xsum_34744458`, an unusual human-interest story about a skip-hire firm returning cremated remains, both contain first-person quotations from named individuals, a stylistic marker log-likelihood correctly associates with human authorship but which curvature's perturbation-based signal does not appear to weight appropriately. Article `xsum_37805755`, on Welsh food-hygiene labelling policy, follows the same pattern. Across all five examples, the shared characteristic is direct attribution: named sources, quoted speech, and specific, verifiable detail. We suspect curvature's perturbation-based scoring is less sensitive to this stylistic marker than log-likelihood's direct token-probability signal, since masking and reconstructing spans within a quotation or a proper name may not reliably reproduce the same named entity or exact wording, artificially inflating the discrepancy score for text that is, in fact, human-written. This is consistent with curvature's comparatively low precision (0.6226, Section 4.2) and its confusion matrix (60 false positives out of 100 human samples, and suggests curvature's false-positive-prone behaviour reflects a systematic sensitivity to this style of factual, attribution-heavy writing rather than isolated edge cases. The remaining disagreements are more mixed: in 9 cases curvature correctly identifies human text that log-likelihood misclassifies as machine-generated, and in 5 cases the detectors disagree on genuinely machine-generated text; the full breakdown is available in our supplementary material.

4.5 Extension: Ensemble and Paraphrasing Robustness Results

Table 6: Ensemble AUROC compared to individual detectors, clean and under a (length-confounded) paraphrasing attack.

Detector	Clean AUROC	Attacked AUROC	Δ
Log-Likelihood	0.9401	0.2289	-0.7112
Curvature	0.6418	0.0029	-0.6389
Ensemble	0.9380	0.1747	-0.7633

Attacked AUROC values are confounded by unintended text compression during paraphrasing (Section 6) and should not be read as a clean robustness result.

5 Discussion

Our results directly address the research question. Using the same scoring backbone, dataset, and evaluation protocol, per-token log-likelihood performed substantially better than curvature-based scoring on the primary test set (AUROC 0.9524 vs. 0.6668). It also achieved a higher AUROC on the secondary evaluation pool (0.9401 vs. 0.6418). This differs from the original DetectGPT evaluation, where curvature-based detection performed better than direct likelihood thresholding [7]. However, this result should be considered alongside later work showing weaker performance in more realistic settings. For example, DetectRL reported an AUROC of only 12.21% for DetectGPT on news writing under real-world conditions, which is much lower than the results reported on controlled benchmarks [14]. Our result for a similar news domain falls between these two reported results: it is higher than the real-world DetectRL result but lower than the performance reported by DetectGPT under controlled conditions. These comparisons are only intended as context rather than direct comparisons, since the studies use different generators, datasets, and evaluation protocols.

One possible reason for the weaker performance of curvature-based scoring is that we used T5-base instead of the proposed T5-large as the perturbation model (Section 6). To test this explanation directly, we additionally ran the full curvature pipeline with the originally proposed T5-large perturbation model on the same test set. This yielded an AUROC of 0.6937 (95% CI: [0.6156, 0.7737]), only a marginal improvement over T5-base's 0.6668, with substantially overlapping confidence intervals and an identical confusion matrix at the selected operating point. Perturbation model size therefore does not appear to be the primary driver of curvature's underperformance in this setting, at a considerable computational cost: approximately 5.4 hours for T5-large's validation and test scoring, versus roughly 35 minutes for T5-base.

This is consistent with the broader curvature literature, which suggests the approach performs best under specific conditions our setup does not fully satisfy. Mitchell et al. [7] report curvature's strongest results when the scoring model closely approximates the generating model's own distribution, since the method's theoretical justification rests on machine-generated text occupying local maxima of *the generator's own* probability landscape, not necessarily that of an unrelated scoring model. In our setup, Llama 3.2 3B scores text generated by a different model, Gemma 3 4B, a scorer-generator mismatch that log-likelihood thresholding is comparatively more robust to, since it only requires the scoring model to assign systematically different average probabilities to human and machine text, not to recover the generator's specific local geometry. This offers a more likely explanation for curvature's underperformance here than perturbation quality alone, and suggests curvature-based detection may be better suited to settings where the scoring and generating models are the same or closely related, consistent with the original DetectGPT evaluations [7], which predominantly scored text using the same model family that generated it. The precision of curvature (0.6226), together with its near-perfect recall (0.9900), also suggests that the detector is frequently classifying human text as machine-generated. This may indicate that the discrepancy signal is noisy or not well calibrated, rather than showing that the underlying approach has no useful information.

This leads to an important practical question: does the additional computational cost of curvature result in better detection? Curvature generates and scores 30 perturbations per sample against log-likelihood's single forward pass, requiring roughly 35 minutes of GPU time for validation and test scoring compared to a small fraction of that for log-likelihood, a cost that did not translate into better detection here. This concern is also reflected in later work. Bao et al. introduced Fast-DetectGPT specifically to address the computational cost of DetectGPT by using a more efficient approximation of the curvature calculation [1]. Our results similarly suggest that the cost of standard perturbation-based curvature scoring is difficult to justify when a much cheaper method performs better.

The secondary ensemble analysis gives a similar result. Combining log-likelihood and curvature using logistic regression produced a high AUROC of 0.9380, but this was still slightly lower than log-likelihood alone (0.9401). This suggests that the two signals do not provide much complementary information in this setting. Combining a strong detector with a weaker detector does not necessarily improve performance, particularly when the weaker detector mainly adds noise rather than useful independent information.

For comparison across the different parts of this project, we also evaluate a TF-IDF logistic regression baseline. We consider two variants. The first was trained separately on a shared dataset generated using ChatGPT, GPT-4, GPT-2, Mistral, and Llama, and then applied without modification to our held-out test set. This provides a genuine domain-transfer evaluation because none of these generators is Gemma 3 4B, which is the generator used in this study. The shared baseline achieved an AUROC of 0.9719, which is close to the 0.9524 achieved by log-likelihood and substantially higher than curvature. Its precision (0.9740) was much higher than its recall (0.7500), suggesting that under domain transfer it became more conservative: it correctly identified most of the text it flagged as machine-generated but missed a larger proportion of the actual machine-generated samples.

The second variant is a TF-IDF baseline trained directly on our XSum/Gemma splits. It achieved a near-ceiling AUROC of 0.9962 (Table 3). This result is not unexpected because, unlike the zero-shot detectors used as our primary contribution, this classifier was trained directly on labelled examples of Gemma 3 4B's output. It can therefore learn lexical patterns specific to both the generator and the dataset that the zero-shot methods cannot use because they do not have access to these labels. The difference between the

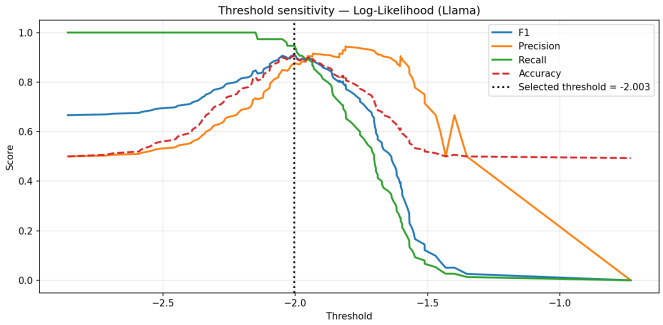


Figure 6: Precision, recall, F1, and accuracy for log-likelihood across candidate validation thresholds, with the selected threshold (−2.0031) marked. Moving the threshold left increases recall at the cost of precision, and vice versa.

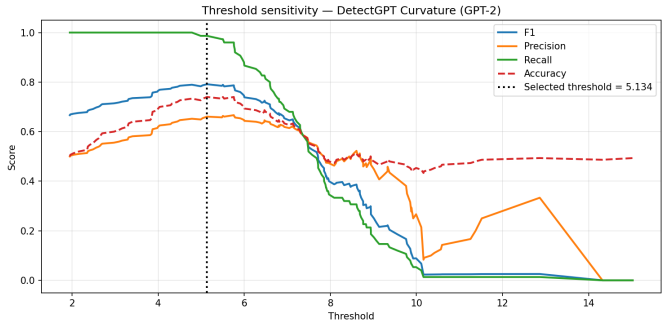


Figure 7: Precision, recall, F1, and accuracy for curvature across candidate validation thresholds, with the selected threshold (5.1340) marked. Curvature's wide gap between precision and recall at its selected threshold is visible here as an asymmetric trade-off curve.

two TF-IDF results—0.9962 in-domain compared with 0.9719 under domain transfer to unseen generators shows that the performance of a supervised lexical classifier can be affected by distributional changes, even when its overall performance remains strong. This result therefore provides evidence that Gemma 3 4B has detectable lexical patterns in this dataset, rather than showing that supervised methods are always better than zero-shot detection. In real-world settings where generators and writing styles can change over time, zero-shot methods have the advantage that they do not need to be retrained for each new generator.

5.1 Threshold Sensitivity

Both detectors reduce a continuous score to a binary decision via a single validation-calibrated threshold, selected using Youden's J statistic to jointly maximise sensitivity and specificity. This choice is not neutral: moving the threshold in either direction directly reshapes the trade-off between false positives (human text incorrectly flagged as machine-generated) and false negatives (machine-generated text that goes undetected), and the two detectors in this study occupy different positions along that trade-off at their selected operating points.

At its chosen threshold, curvature achieves almost perfect recall (0.9900) but has relatively low precision (0.6226). This means that it identifies almost all machine-generated samples, but also incorrectly labels a considerable number of human-written samples as machine-generated. Increasing the curvature threshold would likely reduce these false positives and improve precision, but it would also lower recall because some machine-generated texts that are currently detected would fall below the higher threshold. Log-likelihood, on the other hand, operates at a more balanced point, with a precision of 0.8276 and recall of 0.9600. It is therefore less likely to classify text as machine-generated, leaving less need for a large threshold adjustment.

This difference is important when considering how these detectors might be used in practice. The best threshold depends on which type of error is considered more serious. For example, in academic integrity screening, incorrectly accusing a human of using AI could have serious consequences. A higher threshold would therefore be preferable because it would place more emphasis on precision, even if this means missing some machine-generated text. In contrast, a system used to screen large amounts of content for misinformation might place more importance on finding machine-generated content, even if this results in more false positives. Since curvature currently favours recall, using it in a setting where false positives are particularly costly could result in too many human-written texts being incorrectly flagged. This also supports concerns raised in previous research that the performance of zero-shot detectors can change when they are applied to data that differs from the data used for calibration [14]. Therefore, AUROC alone does not fully describe how a detector will behave in practice, since the chosen threshold can have a substantial effect on the types of errors it makes.

6 Limitations

Dataset scope. Our proposal originally specified Common Crawl, but we use XSum instead to keep all human-written text within a single journalistic domain, at the cost of narrower generalisability beyond news writing. Our dataset also comprises only 500 samples (200 in the held-out test set), considerably smaller than benchmarks used elsewhere in the literature; while our bootstrap confidence intervals account for sampling uncertainty at this scale, individual sample-level differences can noticeably shift reported metrics, and a larger dataset would provide more reliable estimates. All machine-generated text originates from a single generator, Gemma 3 4B, scored with a single backbone, Llama 3.2 3B; results may not generalise to other generator or backbone combinations [6].

Perturbation model and scorer-generator mismatch. We substitute T5-base for the proposed T5-large in our primary curvature experiments; a follow-up re-run with T5-large yielded only a marginal improvement (AUROC 0.6937 vs. 0.6668), indicating perturbation model size is not the primary cause of curvature's underperformance. We instead attribute this to a mismatch between our scoring backbone and generator, discussed in Section 5; curvature-based detection may perform better when the scoring and generating

models are the same or closely related, as in DetectGPT's original evaluations [7].

Paraphrasing attack confound. We evaluated both detectors' robustness by paraphrasing each machine-generated test sample using a pre-trained T5-based paraphrase model and re-scoring the resulting text. This paraphraser substantially compressed document length rather than preserving it, confounding the observed collapse in detector performance with a text-length effect rather than isolating paraphrasing robustness alone. A length-preserving, sentence-level implementation is needed for a clean measurement and is left to future work.

Curvature formula correction. An initial implementation of the curvature discrepancy score omitted standard-deviation normalisation (AUROC 0.6169); we identified and corrected this prior to reporting final results (final AUROC 0.6668), using the formulation matching DetectGPT's original specification [7]. We report this for transparency.

7 Conclusion

This paper presented a controlled, paired comparison of per-token log-likelihood and curvature-based zero-shot detection of LLM-generated text, evaluated under a shared scoring backbone, dataset, and evaluation protocol. Our central finding is clear: log-likelihood substantially and consistently outperformed curvature-based scoring, achieving an AUROC of 0.9524 against curvature's 0.6668 on our held-out test set, a gap that held across an independent secondary evaluation as well (0.9401 vs. 0.6418). Given curvature's roughly 30-fold increase in computational cost relative to log-likelihood, our results do not support its use over the simpler baseline in this setting. A secondary ensemble combining both signals achieved a high absolute AUROC (0.9380) but did not exceed log-likelihood alone, indicating the two signals are not meaningfully complementary here.

These findings should be read within the scope of our experimental design: a single generator (Gemma 3 4B), a single scoring backbone (Llama 3.2 3B), a T5-base perturbation model substituted for the originally proposed T5-large, and an English-language news-domain dataset. An exploratory paraphrasing-attack evaluation was also confounded by unintended text compression and is not reported as a robustness finding (Section 6). We directly tested one candidate explanation for this gap by re-running curvature with the originally proposed T5-large perturbation model; the resulting improvement was marginal (AUROC 0.6937 vs. 0.6668), suggesting the gap more likely reflects a mismatch between the scoring backbone and the generator model rather than perturbation quality alone (Section 5). Future work should implement a length-preserving, sentence-level chunked paraphrasing attack for a clean robustness measurement, and extend the comparison to additional generator models and domains. A larger dataset would also improve the reliability of our estimates, particularly for curvature, where individual sample-level differences visibly influence reported metrics at this scale; however, based on the literature, we do not expect a larger dataset alone to substantially change curvature's relative ranking against log-likelihood. Prior work reports strong curvature performance primarily under settings where the scoring and generating models are closely matched [7], a condition our controlled, cross-model design deliberately does not satisfy; we consider this scorer-generator relationship, rather than dataset scale, the more likely determinant of our result.

A further extension would be to evaluate detectors at sub-article granularity rather than treating each article as a single scoring unit. Since both detectors respond primarily to local statistical or stylistic properties of text rather than to document-level topic or context structure, an article could instead be partitioned into smaller contiguous spans, for example fixed-length windows independent of topical or paragraph boundaries, and each span scored individually. For a test set of 100 articles, splitting each into 20 comparable spans of 50 - 100 words would yield 2,000 scored units rather than 100, providing finer statistical resolution and allowing an analysis of *where* within an article the detection signal is strongest or weakest, rather than only its aggregate value. This is consistent with our framing of detection as a fundamentally stylistic rather than topical task, and would not require restricting the extension to any particular content domain.

Acknowledgements

The author acknowledges the use of Claude AI as an assistance tool during the development of this paper. Claude was used to assist with debugging, rewording and improving the clarity of written sections, and supporting the interpretation and inference of results. All research decisions, analyses, interpretations, and final content were reviewed and verified by the author. The author also acknowledge the use of Google colab Pro for the running of code through the T4 Graphics Processing Unit

References

[1] Bao, G., Zhao, Y., Teng, Z., Yang, L. and Zhang, Y. (2024). Fast-DetectGPT: Efficient Zero-Shot Detection of Machine-Generated Text via Conditional Probability Curvature. In *Proceedings of the 12th International Conference on Learning Representations (ICLR 2024)*, Vienna, Austria. https://arxiv.org/abs/2310.05130

[2] Gehrmann, S., Strobelt, H. and Rush, A. (2019). GLTR: Statistical Detection and Visualization of Generated Text. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics: System Demonstrations (ACL 2019)*, Florence, Italy, pp. 111–116. https://aclanthology.org/P19-3019/

[3] Gemma Team, Google DeepMind (2025). Gemma 3 Technical Report. *arXiv preprint*, arXiv:2503.19786. https://arxiv.org/abs/2503.19786

[4] Ippolito, D., Duckworth, D., Callison-Burch, C. and Eck, D. (2020). Automatic Detection of Generated Text is Easiest When Humans are Fooled. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL 2020)*, Online, pp. 1808–1822. https://aclanthology.org/2020.acl-main.164/

[5] Krishna, K., Song, Y., Karpinska, M., Wieting, J. and Iyyer, M. (2023). Paraphrasing Evades Detectors of AI-Generated Text, but Retrieval is an Effective Defense. In *37th Conference on Neural Information Processing Systems (NeurIPS 2023)*, New Orleans, USA. https://arxiv.org/abs/2303.13408

[6] Macko, D., M'oro, R., Uchendu, A., Lucas, J.S., Yamashita, M., Pikuliak, M., Srba, I., Le, T., Lee, D., Simko, J. and Bielikov'a, M. (2023). MULTITuDE: Large-Scale Multilingual Machine-Generated Text Detection Benchmark. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP 2023)*, Singapore, pp. 9960–9987. https://aclanthology.org/2023.emnlp-main.616/

[7] Mitchell, E., Lee, Y., Khazatsky, A., Manning, C.D. and Finn, C. (2023). DetectGPT: Zero-Shot Machine-Generated Text Detection Using Probability Curvature. In *Proceedings of the International Conference on Machine Learning (ICML 2023)*, Honolulu, Hawaii, pp. 24950–24962. https://arxiv.org/abs/2301.11305

[8] Narayan, S., Cohen, S.B. and Lapata, M. (2018). Don't Give Me the Details, Just the Summary! Topic-Aware Convolutional Neural Networks for Extreme Summarization. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP 2018)*, Brussels, Belgium, pp. 1797–1807. doi: 10.18653/v1/D18-1206

[9] Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W. and Liu, P.J. (2020). Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *Journal of Machine Learning Research*, Vol. 21, No. 1, pp. 5485–5551. https://arxiv.org/abs/1910.10683

[10] Sadasivan, V.S., Kumar, A., Balasubramanian, S., Wang, W. and Feizi, S. (2023). Can AI-Generated Text be Reliably Detected? *arXiv preprint*, arXiv:2303.11156. https://arxiv.org/abs/2303.11156
[11] Solaiman, I., Brundage, M., Clark, J., Askell, A., Herbert-Voss, A., Wu, J., Radford, A., Krueger, G., Kim, J.W. and Kreps, S. (2019). Release Strategies and the Social Impacts of Language Models. *arXiv preprint*, arXiv:1908.09203. https://arxiv.org/abs/1908.09203
[12] Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozi'ere, B., Goyal, N., Hambro, E., Azhar, F., Rodriguez, A., Joulin, A., Grave, E. and Lample, G. (2023). LLaMA: Open and Efficient Foundation Language Models. *arXiv preprint*, arXiv:2302.13971. https://arxiv.org/abs/2302.13971
[13] Uchendu, A., Ma, Z., Le, T., Zhang, R. and Lee, D. (2021). TURINGBENCH: A Benchmark Environment for Turing Test in the Age of Neural Text Generation. In *Findings of the Association for Computational Linguistics: EMNLP 2021*, Punta Cana, Dominican Republic, pp. 2001–2016. https://aclanthology.org/2021.findings-emnlp.172/
[14] Wu, J., Zhan, R., Wong, D.F., Yang, S., Yang, X., Yuan, Y. and Chao, L.S. (2024). DetectRL: Benchmarking LLM-Generated Text Detection in Real-World Scenarios. In *38th Conference on Neural Information Processing Systems (NeurIPS 2024)*, Datasets and Benchmarks Track. https://arxiv.org/abs/2410.23746
[15] Wu, J., Yang, S., Zhan, R., Yuan, Y., Wong, D.F. and Chao, L.S. (2025). A Survey on LLM-Generated Text Detection: Necessity, Methods, and Future Directions. *Computational Linguistics*, Vol. 51, No. 1. doi: 10.1162/coli_a_00549.

A Appendix

A.1 Full Hyperparameter Configuration

The following configuration specifies the model, perturbation, threshold selection, and evaluation parameters used in our experiments.

Table 7: Experimental configuration. Initial Run

Component	Parameter	Setting
Scorer	Model	`Llama-3.2-3B`
	Device	`auto`
	Data type	`float32`
Perturbation	Mask model	`t5-base`
	k	30
	Mask ratio	0.15
Threshold	Strategy	`youden_j`
	Split	`validation`
Evaluation	Bootstrap samples	1000
	Bootstrap seed	42
	Minimum tokens	10

A.2 Full Confusion Matrices

Full confusion matrices corresponding to the summary in Section 4.2.

Table 8: Experimental configuration. Secondary Run

Component	Parameter	Setting
Scorer	Model	`Llama-3.2-3B`
	Device	`auto`
	Data type	`float32`
Perturbation	Mask model	`t5-large`
	k	30
	Mask ratio	0.15
Threshold	Strategy	`youden_j`
	Split	`validation`
Evaluation	Bootstrap samples	1000
	Bootstrap seed	42
	Minimum tokens	10

Table 9: Confusion matrices for the evaluated detection methods. Values represent the percentage of samples classified into each category.

Method / Actual Class	Predicted Machine	Predicted Human
Log-Likelihood		
Actual Machine	96	4
Actual Human	20	80
Curvature		
Actual Machine	99	1
Actual Human	60	40
TF-IDF (Self-trained)		
Actual Machine	93	7
Actual Human	1	99
TF-IDF (Shared, Domain-transfer)		
Actual Machine	75	25
Actual Human	2	98

A.3 Additional Plots

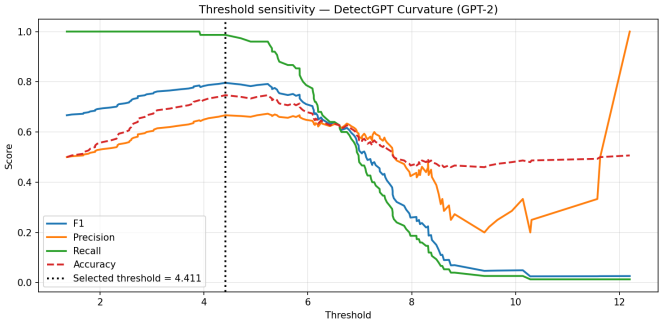


Figure 8: Threshold sensitivity Curvature detector (T5-large perturbation model, follow-up test).

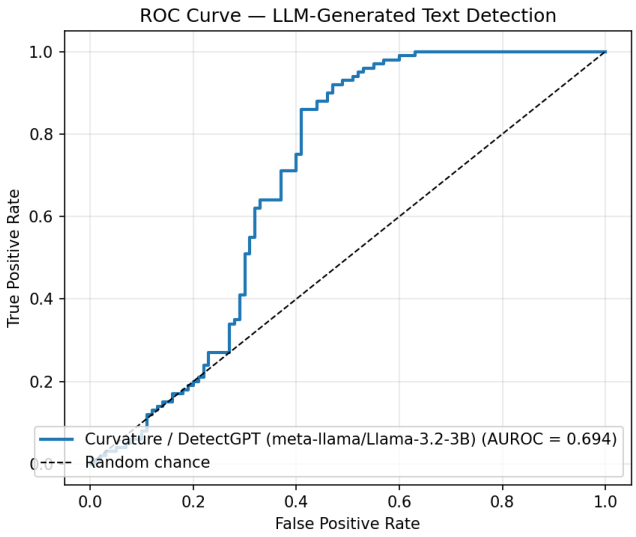


Figure 9: ROC curve Curvature detector (T5-large), compared against the T5-base result in Figure 4.

A.4 Perturbation Model Comparison

Table 10: Curvature detector performance under T5-base vs. T5-large perturbation models.

Model	AUROC	95% CI	Scoring time
T5-base	0.6668	[0.5896, 0.7482]	~35 min
T5-large	0.6937	[0.6156, 0.7737]	~5.4 hrs